

Filejacking: How to make a file server from your browser (with HTML5 of course)

Filejacking: How to make a file server from your browser (with HTML5 of course) - Author not stated, blog.kotowicz.net.

- Published: date not stated
- Original: [<http://blog.kotowicz.net/2011/04/how-to-make-file-server-from-your.html>](http://blog.kotowicz.net/2011/04/how-to-make-file-server-from-your.html)
- Preserved from: <http://blog.kotowicz.net/2011/04/how-to-make-file-server-from-your.html> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Back in the days of [browser wars](#), there was a joke: *Internet Explorer is the only web browser that makes Internet browse your computer*. Through various security flaws, IE was exploitable and allowed for remote code execution that could e.g. steal your sensitive files. But now the times are different. It's not that easy to exploit current browsers, they get patched (relatively) quickly. Attackers cannot easily access your files using browsers vulnerabilities, so they turn to the weakest link - **users**. In this post we'll try to explore what current browsers can do with your files.

Your file, please

How can a website access user's files? Traditionally, user has to **upload the file**. Users commonly share photos, videos upload their files for online conversion tools etc. You could (theoretically) be tricked into uploading a sensitive file into a malicious website ("*please submit your private key for checking it's strength*"), but, seriously, who falls for that?

Lately, [File API](#) allowed Javascript to access the file once it is **chosen** by the user (i.e. before uploading it). Apart from delivering better file uploading experience, it might also be used maliciously [to steal your files in XSS attack](#). Also, with clever styling you can hide input type=file control so that the user is unaware that he's going to upload the file. But still - the only leaked file is the one user chose using 'Open File' dialog.

!

Users are aware of what file uploading is and are reluctant to choose Downloaded Files/nothing here/move along/boring family photos/1/b00bs.jpg when working inside a browser, so it's not a big deal, right?

Wrong. It's 2011, web applications need new features, browsers are hurrying to implement them, sometimes security is an afterthought.

But first, a gift

I've got some gifts for you*. *I gathered some of the latest hacking tricks for all browsers, spiced it up with an algorithm that will send you a ZIP file crafted especially for you based on your answers. Just fill out the short quiz and wait for the file download.

**Update: **I'm currently experiencing traffic spike on the server, generating file might take a while.

**Using Chrome / Chromium **please navigate to:

<http://kotowicz.net/wu/>

and claim your gift :)

Now back to me

input type=file directory is a splendid feature. It allows you to upload contents of a chosen directory. Great when you'd like to submit a gallery of pictures to Facebook, ain't it? Currently, it's [implemented in Webkit \(latest Chrome / Chromium\)](#) - not yet in Safari, Firefox or any other browser.

However it has a problem - **that feature is new to users**, they don't know what are it's consequences, there are no warnings either. For all they know, **they're just selecting a folder** using the OS native "select folder" dialog. Similar to what happens when e.g. choosing a download destination.

While you probably didn't fell for my trick, most users are not that smart nor security-minded. And the consequences of sharing a folder are **much worse** than of sharing a single file. Don't believe me? See

<http://attacker.kotowicz.net/wu/evil.html> - the backend of the service. Or just look at the video:

File server inside your browser

The given example is another example of [UI redressing attack](#) exploiting new features of HTML5. The elements of the scam:

- a phishing site with "hacking tricks" bait
- transparent input type=file directory over the fake download button
- launching another window to perform real work (to survive closing initial window by the user)
- the new window sends the file list from the chosen directory to the server
- additionally, it uploads one sample image, if it finds one in your directory
- .. and polls the server repeatedly for further commands
- server control panel gets the list of connected clients and their files

- server operator can choose the files to download
- requests for new files reach the clients, and they send the files back

Your browser has now become a file server, serving files from your chosen directory. More features follow!

- cross domain
- easily served through XSS vulnerability
- server/client could be automated to e.g. send all Excel files at once.
- and, it's HTML5 compatible

Nifty!

Brave new world?

Current web applications demand more power from browsers. With features like

- directory upload,
- offline storage,
- drag & drop support
- extensive styling
- audio & video support
- WebSockets
- notifications

they're getting closer to desktop applications each year. Granted, they all run in a browser "sandbox" with its security policies.

However, *users* are not aware of what current browser can do, so they can be tricked into running the malicious app. And, with XSS being so popular, malicious app may be pretty much every site on the Internet.

Browser vendors try to educate users and prevent them from choosing unsafe settings (Geolocation bar is an example). Shouldn't similar 'warning' be displayed when using input type=file directory ? After all, it's only one click away and the risks of sharing a whole directory are huge. So, WebKit, what do you think?